



MVP CONF

2026

Do Diagnóstico à Otimização

Tuning de modelos semânticos no Microsoft Fabric

André Rosa

Microsoft Certified Trainer

www.mvpconf.com.br



Nossos Patrocinadores

2026



QUEM CONDUZ A SESSÃO

Experiência que conecta estratégia, tecnologia e ensino

**20+**

anos em gestão e consultoria de TI

**20 mil+**

horas em projetos de tecnologia

**105 mil+**

alunos em 107 países

FORMAÇÃO

Sistemas de Informação
Gestão de TI
Pós-graduação em BI
Pós-graduação em BI com Big Data

EXPERIÊNCIA

Gestão e consultoria de TI há mais de 20 anos
ERP SAP e TOTVS (Logix)
Projetos em empresas nacionais e multinacionais

SPEAKER

André Rosa

Head of Data & AI e Tech Learning
DataEX / IT Forest
Udemy Instructor Partner
Microsoft Certified Trainer (MCT)

Tecnologia aplicada. Ensino que escala.

CERTIFICAÇÕES

DP-700

DP-600

DP-203

DA-100

PL-300

PL-400

DP-300

DP-500

AZ-900

DP-900



LINKEDIN

ANDRE-ROSA77

Escaneie e conecte-se



Todo mundo já viveu essa cena

 **18s**

para o relatório executivo
abrir

9 em 10

usuários abandonam
antes do 2º filtro

1 pessoa

sabe explicar
o que a medida faz

O reflexo é culpar a capacidade. Mas o problema quase sempre está no modelo.



Aumentar o SKU
compra tempo



Corrigir o modelo
resolve a causa e custa menos

Performance, governança e experiência do usuário pagam a conta.

O modelo semântico é o coração da análise de dados



Coração comprometido

Bombeia com esforço: cada consulta varre colunas que ninguém usa.

Relatório trava, refresh estoura a janela, capacidade no limite.



Coração saudável

Star schema, colunas necessárias, medidas limpas: menos esforço por consulta.

Mesmos dados, relatório em segundos e capacidade sobrando.

Coração = modelo semântico · sangue = dados · corpo = relatórios

Aumentar a capacidade é dar mais sangue a um coração doente: alivia por um tempo e não corrige a causa.

O percurso de hoje

1



Diagnosticar

Do sintoma à causa raiz.

2



Instrumentar

VertiPaq, BPA e uso real com o Labs.

3



Acelerar

O Copilot lê a evidência e propõe a correção.

4



Corrigir

As cinco camadas de tuning e a prova do ganho.

5



Sustentar

Fluxo ponta a ponta e governança contínua.

3 blocos guiados, com telas e saídas reais · base de dados, notebooks e código saem daqui com você

O sintoma engana. A causa quase nunca está onde dói.

SINTOMA VISÍVEL

Relatório lento ao abrir

Refresh que estoura a janela

Modelo gigante na memória

Números que não batem

Copilot respondendo mal

Capacidade sempre no limite

CAUSA RAIZ MAIS COMUM

Visuais demais + medidas com iteradores aninhados

Colunas calculadas e tabelas de data automáticas

Colunas de alta cardinalidade que ninguém usa

Relacionamento bidirecional e granularidade errada

Sem descrições, sem nomes de negócio, campos expostos

Modelo Direct Lake caindo em fallback para DirectQuery

Nenhuma dessas causas se enxerga pelo sintoma — é por isso que o diagnóstico precisa de instrumento, não de palpite

A base da sessão: Contoso Vendas

LINHAS

5.000.000

TABELA

1 (flat)

COLUNAS

44

PARQUET

1,25 GB

PERÍODO

2023–2025

FATURAMENTO

R\$ 7,64 bi

Gerador em Python no material da sessão.

A semente é fixa: a sua base sai idêntica à minha —
e --linhas ajusta o volume à sua capacidade.

O que ela tem de errado — de propósito



- 3 colunas GUID — 5 milhões de valores distintos cada
- 4 colunas datetime com segundos e microssegundos
- JSON bruto e observação em texto livre
- 17 campos de dimensão desnormalizados no fato
- 5 colunas recalculáveis em DAX
- Status do pedido com 12 grafias diferentes

Base sintética, sem dado real — mas com os defeitos que você encontra em produção. É ela que gera todas as saídas que você vê hoje.

O kit de diagnóstico — e quando usar cada peça

- | | | |
|---|--|-----------------------|
| 1 | Um visual está lento? | Performance Analyzer |
| 2 | Onde a memória está indo? | DAX Studio / VertiPaq |
| 3 | Quais consultas doem em produção? | Workspace Monitoring |
| 4 | Quem consome Capacity Units na capacidade? | Capacity Metrics App |
| 5 | Quero isso no notebook, sem instalar nada? | Semantic Link (SemPy) |
| 6 | Quero tudo isso em Python, em escala? — o foco de hoje | Semantic Link Labs |

```
In [1]: python · setup

1 %pip install semantic-link-labs==0.17.0
2 import sempy_labs as labs
3 from sempy_labs.tom import connect_semantic_model
```

O Labs não substitui o Performance Analyzer. Ele resolve o que não escala: 40 modelos, toda semana. É open source e muda rápido, fixe a versão.

Semantic Link lê o modelo. O Labs age sobre ele.

1 Semantic Link (SemPy)

- 1 **First-party** já no runtime do Fabric, nada a instalar
- 2 **Leitura** `read_table`, `evaluate_dax`, `list_measures`
- 3 **pandas e Spark** com a semântica preservada
- 4 **Suporte oficial** documentação no Microsoft Learn

Quando usar: ler dados e metadados do modelo no notebook.

2 Semantic Link Labs (sempy_labs)

- 1 **Open source** `%pip install semantic-link-labs`
- 2 **Sobre o SemPy** centenas de funções de diagnóstico
- 3 **Funções-chave** `vertipaq_analyzer`, `run_model_bpa`, `TOM`
- 4 **Acesso antecipado** evolui rápido, sem SLA de produto

Quando usar: medir, comparar antes e depois e aplicar a correção.

Regra prática: o SemPy é a leitura suportada pelo produto; o Labs é o diagnóstico e a escrita, em acesso antecipado. Em produção, prenda a versão do Labs no ambiente do workspace.

Na prática os dois andam juntos: o SemPy conecta, o Labs mede, aponta e aplica a correção.

Passo 1 — VertiPaq Analyzer: o raio-X da memória

```
In [2]: python · sempy_labs
```

```
1 labs.vertipaq_analyzer(  
2     dataset="Contoso Vendas",  
3     workspace="Analytics PRD",  
4     read_stats_from_data=True  
5 )
```

O que ele devolve



- Tamanho por tabela, coluna, hierarquia e relacionamento
- Cardinalidade e tamanho de dicionário por coluna
- Relacionamentos com risco de violação de integridade
- Partições, segmentos e contagem de linhas

Onde olhar primeiro

- ▶ **Colunas > 5% do modelo**
na nossa base, 4 colunas somam 57%
- ▶ **Alta cardinalidade**
GUID, datetime com segundos, ID de linha
- ▶ **Dictionary size alto**
texto livre que ninguém filtra
- ▶ **Tabelas de data automáticas**
uma por coluna de data — silenciosas
- ▶ **Relacionamento many-to-many**
custo de consulta e ambiguidade

`read_stats_from_data=True` lê estatísticas direto do modelo — mais preciso e mais caro. Use `export="table"` para gravar o resultado no lakehouse.

Passo 2 — Best Practice Analyzer: 64 regras em uma célula

In [3]: python · BPA

```
1 labs.run_model_bpa(  
2     dataset="Contoso Vendas",  
3     extended=True, language="pt-BR")
```

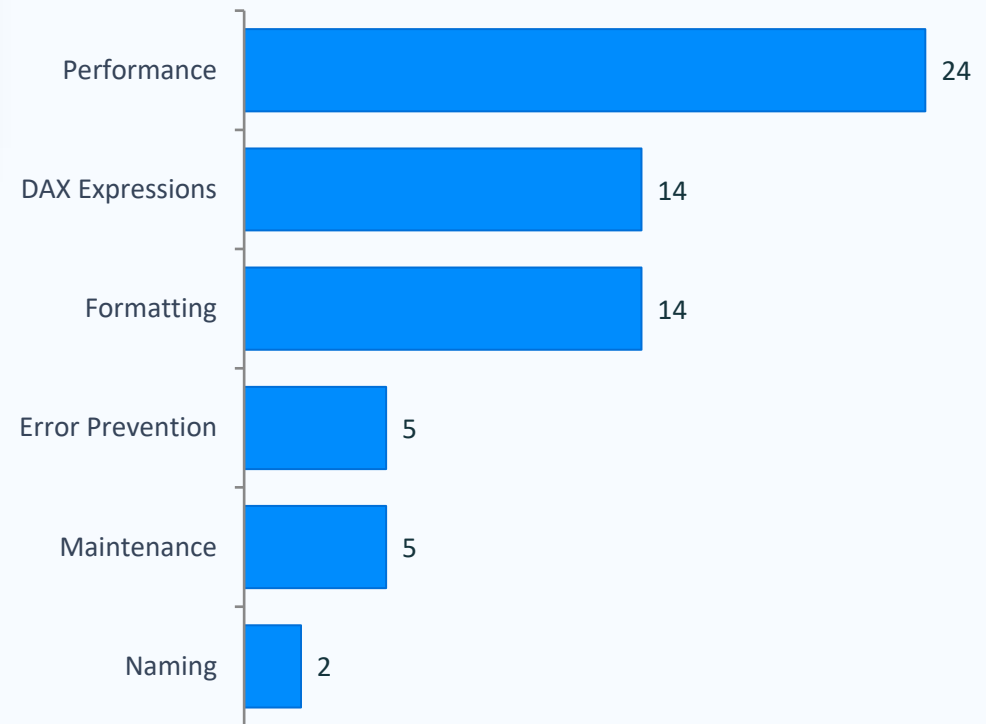
extended=True cruza as regras com as estatísticas do VertiPaq — você vê o **impacto em MB**, não só a violação.

Também vale conhecer



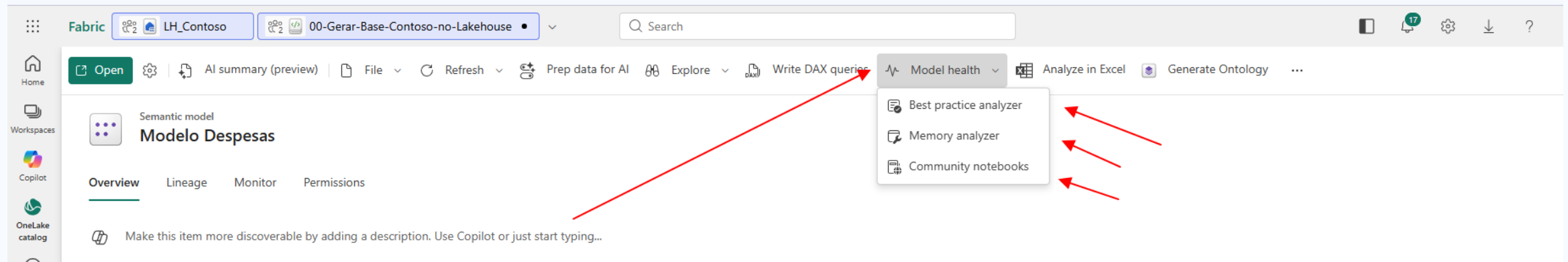
- `run_model_bpa_bulk()` — o workspace inteiro de uma vez
- `create_model_bpa_semantic_model()` — histórico em relatório
- `model_bpa_rules()` — regras próprias da sua empresa

As 64 regras por categoria



Regras padrão do Semantic Link Labs 0.17.0 · quase 40% delas são de performance

O BPA também roda fora do notebook, em Model health



Best practice analyzer

As mesmas 64 regras do run_model_bpa, em tela.

Memory analyzer

O VertiPaq Analyzer na interface: tamanho por tabela e coluna.

Community notebooks

Notebooks prontos do Semantic Link Labs, a um clique.

Caminho: abra o modelo semântico → Model health. A tela é boa para explorar; o notebook é o que agenda, versiona e compara antes x depois.

Duas células e o BPA roda no seu modelo

Import the Semantic Link library

```
1 import sempy.fabric as fabric
2
3 dataset = "Contoso Vendas v2 (tunado)" # Enter the name or ID of the semantic model
4 workspace = "MVP CONF 2026" # Enter the workspace name or ID in which the semantic model exists
```

[1] ✓ 11 sec - Command executed in 6 sec 817 ms by Andre Antonio da Rosa on 25/09/2026, 17:18:05

Python ▾

Run the Best Practice Analyzer on your semantic model

```
1 fabric.run_model_bpa(dataset=dataset, workspace=workspace)
```

[2] ✓ 21 sec - Command executed in 20 sec 549 ms by Andre Antonio da Rosa on 25/09/2026, 17:18:26

Python ▾

Importe o `sempy.fabric`, aponte `dataset` e `workspace` e chame `run_model_bpa` — a execução leva segundos e já devolve as violações.

O resultado: violações por categoria, regra e objeto



COMECE AQUI · PERFORMANCE

Run the Best Practice Analyzer on your semantic model

+ Code + Markdown

1 `fabric.run_model_bpa(dataset=dataset, workspace=workspace)`
[2] ✓ 21 sec - Command executed in 20 sec 549 ms by Andre Antonio da Rosa on 25/09/2026, 17:18:26

Formatting	Maintenance	Performance		
 (71) +  (26)	 (44)	 (7)		
	Rule Name	Object Type	Object Name	Severity
	Do not summarize numeric columns	Column	'dim_cliente'[cliente_key]	
	Do not summarize numeric columns	Column	'dim_data'[ano]	
	Do not summarize numeric columns	Column	'dim_data'[dia]	
	Do not summarize numeric columns	Column	'dim_data'[mes_numero]	
	Do not summarize numeric columns	Column	'dim_loja'[loja_key]	
	Do not summarize numeric columns	Column	'dim_produto'[custo_padrao]	
	Do not summarize numeric columns	Column	'dim_produto'[preco_lista]	

Formatting		Maintenance		Performance		
 (71) +  (26)		 (44)		 (7)		
Rule Name		Object Type	Object Name		Severity	
Do not use floating point data types		Column	'dim_produto'[custo_padrao]			
Do not use floating point data types		Column	'dim_produto'[preco_lista]			
Do not use floating point data types		Column	'fact_vendas'[custo_total]			
Do not use floating point data types		Column	'fact_vendas'[desconto_pct]			
Do not use floating point data types		Column	'fact_vendas'[preco_unitario]			
Do not use floating point data types		Column	'fact_vendas'[valor_liquido]			
Model should have a date table		Model	Model			

Formatting, Maintenance e Performance separam o ruído do que dói: comece pela aba Performance — são poucas regras, e são as que custam consulta.

O Copilot lê o modelo e sugere a correção

The screenshot displays the Microsoft Power BI Desktop interface. The top ribbon includes tabs for File, Home, and Help. The Home tab is active, showing various toolbars for Data, Queries, Calculations, Parameters, Security, Relationships, Modeling, Model health, Ontology, Explore, and Copilot. The main workspace shows a semantic model with three tables: dim_cliente, dim_loja, and fact_vendas. dim_cliente is connected to fact_vendas with a 1-to-many relationship. dim_loja is also connected to fact_vendas with a 1-to-many relationship. The fact_vendas table is expanded, showing columns: canal, cliente_key, custo_total, data, desconto_pct, forma_pagamento, hora_venda, loja_key, and preco_unitario. The Copilot sidebar on the right is open, showing the Copilot logo and the text 'Uncover insights with the help of AI'. Under the 'Recommended' section, there are two suggestions: 'Suggest improvements to this semantic model' and 'Suggest improvements to this semantic model'. A green arrow button is visible at the bottom right of the Copilot sidebar.

Com a evidência do BPA e do VertiPaq na mão, peça ao Copilot melhorias no modelo semântico — ele explica a regra, aponta o objeto e propõe o ajuste; a decisão continua sua.

As violações que mais devolvem resultado

- | | | | |
|---|------------------------------------|--|-------|
| 1 | Auto date/time ligado | Cria uma tabela de data oculta por coluna de data. Inchaço puro. | Alto |
| 2 | Colunas de alta cardinalidade | Datetime com hora, GUID e IDs que nenhum visual usa. | Alto |
| 3 | Colunas calculadas no modelo | Compressão pior que a coluna importada. Prefira tartar no ETL, sem ser o Power Query | Alto |
| 4 | Filtro bidirecional / many-to-many | Ambiguidade no filtro e consulta mais cara. | Médio |
| 5 | Iterador desnecessário no DAX | SUMX onde SUM resolve. Multiplica varredura de linha. | Médio |
| 6 | Objetos sem descrição | Governança e qualidade das respostas do Copilot. | Médio |

Impacto estimado sobre memória e tempo de consulta — priorize de cima para baixo

Passo 3 — O que realmente está sendo usado?

A pergunta que ninguém consegue responder



Dá para apagar esta medida sem quebrar nenhum dos relatórios?

- `list_semantic_model_object_report_usage()` — quantas vezes cada objeto é referenciado
- `get_measure_dependencies()` — o que a medida consome
- `measure_dependency_tree()` — a árvore, visualmente
- `list_report_semantic_model_objects()` — campos usados por relatório
- `list_semantic_model_errors()` — referências quebradas

In [4]: python · uso real

```
1 df = labs.list_semantic_model_object_report_usage(  
2     dataset="Contoso Vendas",  
3     include_dependencies=True,  
4     extended=True)
```

```
5 df[df["Report Usage Count"] == 0]
```

Out[4]: objetos sem referência em relatório · 1,1 GB recuperáveis

Por que isso importa

`extended=True` traz o tamanho em memória de cada objeto.

Você passa de "acho que dá para remover" para "remover estas colunas devolve 1,1 GB e não afeta relatório nenhum".

Limitação: a contagem cobre os relatórios do workspace analisado. Excel, Analyze in Excel e apps externos não aparecem — cruze com o Workspace Monitoring antes de apagar.

Passo 4 — Direct Lake: onde o diagnóstico muda de lugar

Direct Lake on OneLake



- Várias fontes Fabric com tabelas Delta
- Suporta modelo composto
- Não faz fallback: se não carregar, a consulta falha

Direct Lake on SQL

- Uma única fonte, via SQL analytics endpoint
- Não suporta modelo composto
- Faz fallback para DirectQuery — e você não é avisado

In [5]: python · Direct Lake

```
1 from sempy_labs import directlake as dl
2 dl.check_fallback_reason(dataset="Contoso DL")
3 dl.get_direct_lake_guardrails()
4 dl.get_directlake_guardrails_for_sku("F64")
```

Guardrails no F64

Arquivos Parquet / tabela	5.000
Row groups / tabela	5.000
Linhas por tabela	1,5 bilhão
Memória do modelo	25 GB

Estourar o guardrail no on SQL não gera erro: gera fallback silencioso e lentidão. Verifique sempre antes de culpar o DAX.

Fonte: Direct Lake overview, Microsoft Learn (valores por SKU; confira a tabela vigente).

RECOMENDAÇÃO · 1

Comece por estas quatro células

- 1 `vertipaq_analyzer()` onde está o peso do modelo
- 2 `run_model_bpa(extended=True)` o que está errado, por impacto
- 3 `list_semantic_model_object_report_usage()` o que ninguém usa
- 4 `check_fallback_reason()` se for Direct Lake

Recomendado rodar este bloco antes de qualquer tuning: é o diagnóstico mínimo de qualquer modelo.

O Copilot escreve o rascunho — a decisão técnica é sua

Preview

Copilot no web modeling

- Renomeia tabelas e colunas
- Cria relacionamentos
- Gera medidas DAX
- Aponta nomenclatura inconsistente

GA

Copilot no notebook



- Explica a saída do VertiPaq/BPA
- Gera o código `sempy_labs`
- Transforma DataFrame em plano
- Documenta a análise

Preview

Copilot / IA no DAX

- Explica medida herdada
- Sugere reescrita
- Propõe variáveis e DIVIDE
- Traduz intenção em DAX



Não delegue

decidir o que pode ser apagado · medir memória e cardinalidade · garantir que o DAX está correto · conhecer a regra de negócio · testar performance · assumir a responsabilidade pela mudança

O Copilot lê evidência e escreve rascunho. A evidência vem do Labs. A decisão continua sendo sua.

O prompt que funciona tem quatro partes

1

Papel

Você é um especialista em tuning de modelos tabulares.

2

Evidência

Cole aqui a saída do BPA / VertiPaq — em texto, não em print.

3

Restrição

Não pode alterar nomes de medidas já usadas em relatórios.

4

Formato

Devolva uma tabela: problema · impacto · ação · risco.

Exemplo real

"Você é especialista em modelos tabulares. Abaixo está a saída do run_model_bpa do modelo Contoso Vendas — 5 milhões de linhas, 44 colunas numa única tabela.

Restrições: não renomear medidas já publicadas e não remover colunas usadas em RLS.

Devolva as 8 ações de maior impacto em memória, como tabela: ação · ganho estimado · risco · como validar."

Print de tela vira prompt ruim. Passe DataFrame como texto ou CSV.

RECOMENDAÇÃO · 2

Transforme evidência em plano de ação

- Exportar a saída do BPA como texto (`df.to_markdown()`)
- Prompt com papel + evidência + restrição + formato
- Conferir cada sugestão contra o VertiPaq — duas estarão erradas
- Copilot no web modeling gerando descrições em lote

Recomendado conferir cada sugestão da IA contra o VertiPaq antes de aplicar qualquer mudança.

Tuning tem cinco camadas — e ordem importa

| 1

Armazenamento

O que entra no modelo

| 2

Modelagem

Como as tabelas se relacionam

| 3

DAX

Como as medidas calculam

| 4

Delta / Direct Lake

Como o arquivo está no lake

| 5

Relatório

O que o visual pede

Otimizar DAX num modelo com 400 colunas inúteis é enxugar gelo. Comece sempre pela camada 1.



Camadas 1 e 2 — onde está o maior retorno

1 Armazenamento



- 1 **Auto date/time OFF** dimensão de data única
- 2 **Colunas sem uso** remova o que o VertiPaq aponta
- 3 **Datetime dividido** data + hora derruba a cardinalidade
- 4 **Decimal → moeda fixa** precisão fixa onde couber
- 5 **Coluna calculada** resolva no ETL
- 6 **Chaves técnicas** oculte em vez de expor

2 Modelagem



- 1 **Star schema de verdade** snowflake só se justificado
- 2 **Filtro bidirecional** use CROSSFILTER pontual
- 3 **Tabela de data** marque-a como tabela de data
- 4 **Many-to-many** quase sempre é dimensão faltando
- 5 **Grupos de cálculo** no lugar de 40 medidas iguais
- 6 **Uma granularidade por fato** sem misturar níveis
- 7 **Medidas explícitas** desencoraje as implícitas

Tudo isso é aplicável via TOM em Python: `connect_semantic_model(readonly=False)` permite automatizar a correção, não só o diagnóstico

Camada 3 — DAX: menos varredura, mais intenção

ANTES

DAX medida · 2 iteradores

```
1 Margem % =  
2 DIVIDE(  
3     SUMX(Vendas,  
4         Vendas[Valor] - RELATED(Produto[Custo])),  
5     SUMX(Vendas, Vendas[Valor])  
6 )
```



Custo calculado no ETL

vira coluna do fato — some, não itera



DIVIDE

trata divisão por zero sem IF

DEPOIS

DAX medida · sem iterador

```
1 Margem % =  
2 VAR _receita = [Receita]  
3 VAR _custo   = [Custo]  
4 RETURN  
5     DIVIDE(_receita - _custo, _receita)
```



Variáveis

avaliam uma vez, não a cada linha



Medidas reaproveitadas

uma definição, um lugar para corrigir



Meça sempre: `labs.get_dax_query_memory_size()` e `labs.get_dax_query_dependencies()` mostram o custo real da consulta

Camadas 4 e 5 — o lake e o relatório também são o modelo

4 Delta / Direct Lake



- 1 **V-Order** ligado nas tabelas gold
- 2 **Particionamento** Ano/Mês quando o filtro permitir pruning
- 3 **OPTIMIZE e VACUUM** na rotina de manutenção
- 4 **Menos arquivos** row groups maiores
- 5 **delta_analyzer()** veja o arquivo por dentro
- 6 **Cache aquecido** warm_direct_lake_cache após o framing
- 7 **View não materializada** evite: causa fallback

5 Relatório

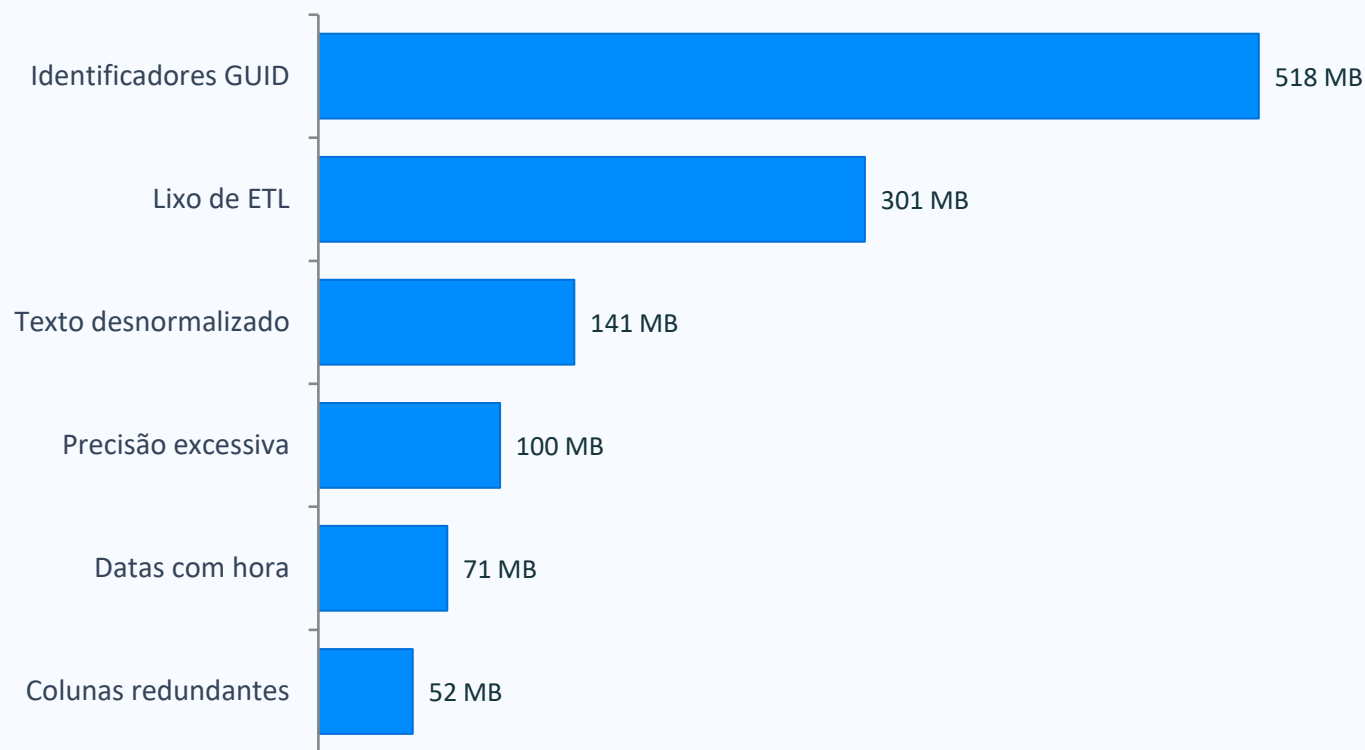


- 1 **Menos visuais** cada visual é uma consulta
- 2 **Tabela de detalhe** evite dezenas de colunas
- 3 **Slicers pesados** viram filtro de painel
- 4 **Botão Aplicar filtros** uma consulta por seleção
- 5 **Top N explícito** no lugar de “mostrar tudo”
- 6 **Interações cruzadas** reduza as desnecessárias
- 7 **Página inicial leve** a primeira impressão é a que fica

Tamanho recomendado: arquivos Delta/Parquet entre 128 MB e 1 GB — ~400 MB com row groups de ~2 mi de linhas atendem bem SQL endpoint e Warehouse; para Direct Lake, V-Order ligado e row groups de 8 mi+ de linhas, com o menor número possível de arquivos.

Fonte: Microsoft Learn — “Tune the size of Delta table data files”, “Understand Direct Lake query performance” e “Cross-workload table maintenance and optimization”.

Contoso Vendas: onde estavam 1,25 GB



MODELO SEMÂNTICO

1,25 GB

→ 106 MB

LARGURA DA TABELA

44 colunas

→ 14 no fato + 5 dimensões

RESULTADO DE NEGÓCIO

R\$ 7,64 bi

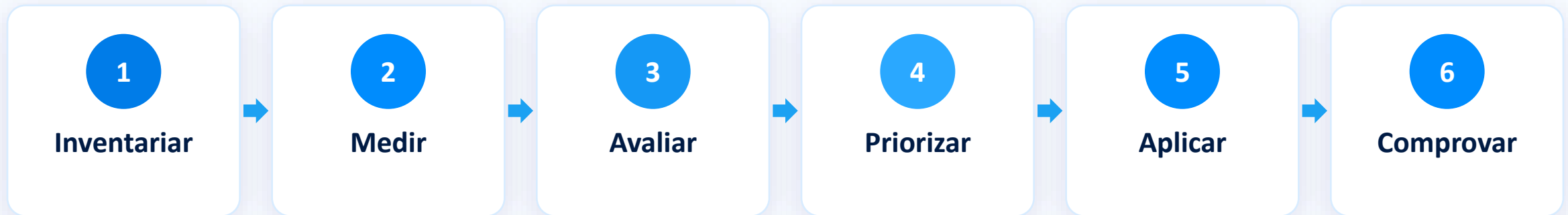
faturamento idêntico

-91,7% de peso

Só 92 MB dos 1,25 GB eram, de fato, dados de negócio.

Medido no Parquet da base de demonstração (5 mi de linhas). O número do modelo semântico sai do vertipaq_analyzer no notebook da sessão — a proporção entre as colunas se mantém.

O fluxo ponta a ponta que você leva pra casa



In [7]: python · o fluxo inteiro em uma célula

```
1 DS = "Contoso Vendas"
2 labs.list_semantic_model_objects(DS)           # 1 inventariar
3 labs.vertipaq_analyzer(DS); labs.delta_analyzer(DS) # 2 medir
4 labs.run_model_bpa(DS, extended=True)          # 3 avaliar
5 connect_semantic_model(DS, readonly=False)     # 5 aplicar
6 labs.get_dax_query_memory_size(DS)             # 6 comprovar
```

Roda inteiro em um notebook. Agende semanalmente e o diagnóstico deixa de ser projeto e vira rotina.

De um modelo para o tenant inteiro

In [6]: python · em lote

```
1 # varre workspaces inteiros
2 labs.run_model_bpa_bulk(
3     workspace=["Analytics PRD", "Financeiro"],
4     extended=True)

5 # histórico consultável em relatório
6 labs.create_model_bpa_semantic_model()
```

O resultado vira produto

Um lakehouse com o histórico de BPA e VertiPaq de todos os modelos. Um relatório de qualidade que o time de BI acompanha como acompanha um SLA.

A rotina que sustenta



Semanal

BPA em lote nos workspaces de produção

Semanal

Consultas mais lentas via Workspace Monitoring

Mensal

VertiPaq nos 10 maiores modelos

Mensal

Objetos com uso zero → fila de remoção

Trimestral

Revisão de descrições, nomes e sinônimos

Service principal + notebook agendado: roda sem ninguém logado.

RECOMENDAÇÃO · 3

Aplique e comprove o ganho

- `connect_semantic_model(readonly=False)` — ocultar e remover em lote
- Gerar descrições nas medidas sem descrição
- Rodar o BPA de novo — a régua antes/depois
- `run_model_bpa_bulk` no workspace inteiro



Nunca aplique direto em produção: faça cópia do modelo, use workspace separado e tenha backup antes.

Mesmas 5 milhões de linhas, 19,5× menos memória

Linhas (granularidade)

5 milhões

mesmas linhas

5.000.000 → 5.000.000

Tamanho em memória

-95%

19,5× menor

1.461,7 MB → 74,8 MB

Arquivos no Delta

-99,5%

200× menos

200 → 1

Colunas materializadas

-68%

3,1× menos

44 → 14

TABLE VIEW

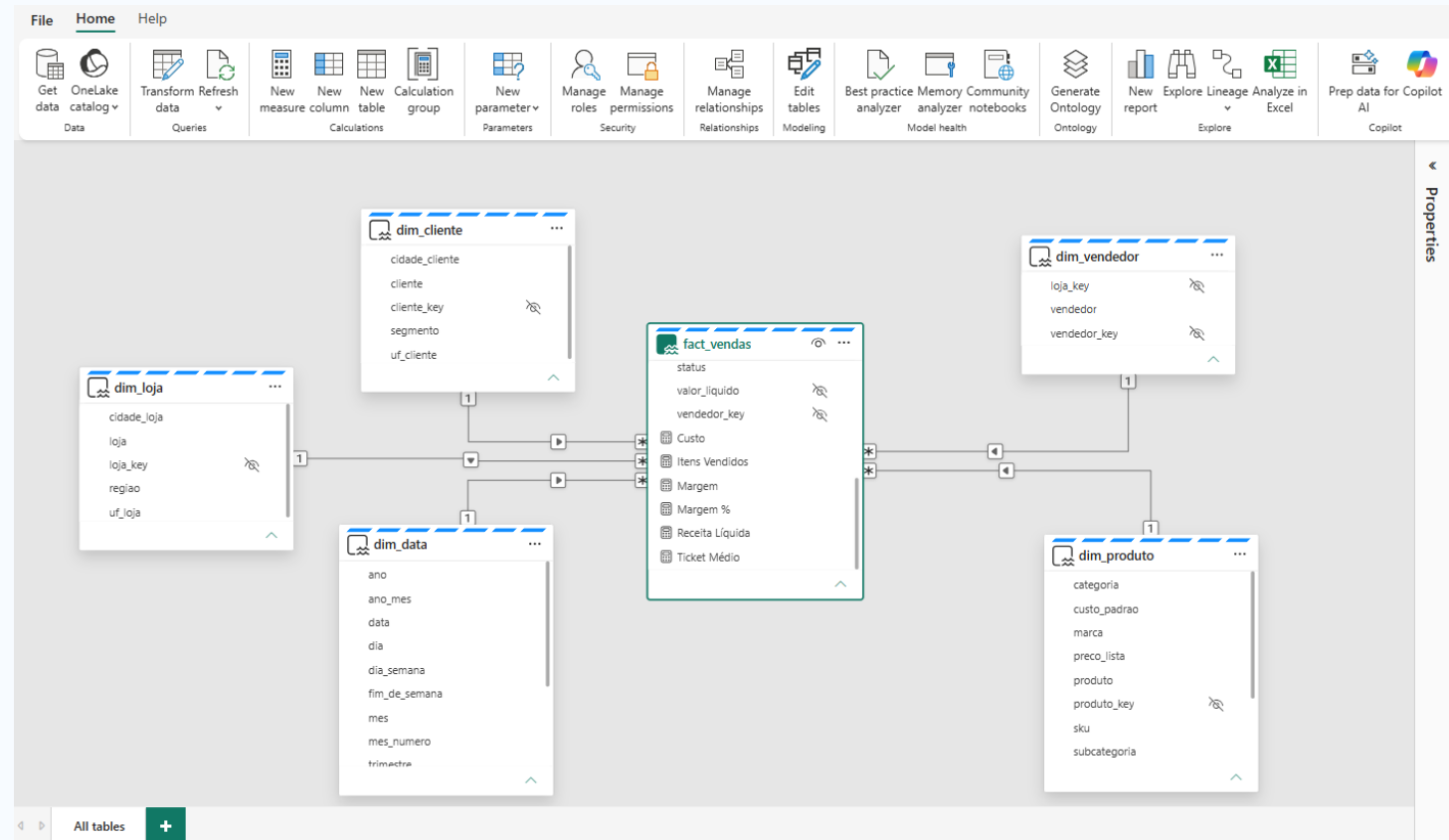
	ABC Tabela	12L Linhas	12L Arquivos	12 Tamanho (MB)	12L Colunas
1	bronze.vendas_flat	5000000	200	1461.7	44
2	gold.fact_vendas	5000000	1	74.8	14

De onde vem o ganho

- Star schema no lugar da tabela flat
- Colunas de alta cardinalidade fora do modelo
- 200 arquivos compactados em um só

delta_analyzer nas duas tabelas do mesmo dado. Menos arquivos corta overhead de leitura; menos colunas corta dicionário no VertiPaq.

O modelo no estado da arte: star schema enxuto



Um fato e cinco dimensões, chaves técnicas ocultas, medidas explícitas e tabela de data marcada — é este o desenho que sustenta performance, governança e boas respostas do Copilot.

Prep data for AI: deixe o modelo pronto para o Copilot

 Prep data for AI ×

Get started

Simplify the data schema

Verified answers

Add AI instructions



Prep this model to be AI-ready

Improve Copilot insights about this semantic model in a few simple steps. Your changes will apply to all Copilot chats, but updates may take a moment. [Learn more](#)

 **Simplify the data schema**

Improve response accuracy by deselecting fields Copilot doesn't need to analyze.

 **Verified answers**

Save predefined responses for your most critical business topics.

 **Add AI instructions**

Help Copilot understand industry terms, business priorities, and important data fields.

Três passos na própria interface: simplificar o esquema (tirar do Copilot os campos que ele não precisa ler), cadastrar respostas verificadas e adicionar instruções de negócio.

www.mvpconf.com.br

O modelo bem tunado é o modelo pronto para IA



Nomes de negócio

"Receita Líquida", não "vlr_liq_2". O Copilot lê o nome antes de tudo.



Descrições

Os primeiros 200 caracteres da descrição vão para o contexto do Copilot.



Campos ocultos

Chave técnica visível é campo que o Copilot vai tentar usar.



Relacionamentos explícitos

Cardinalidade e direção corretas evitam resposta ambígua.



Medidas canônicas

Uma definição de "margem". Não sete variações parecidas.



Valores padronizados

"Faturado" — não as 12 grafias que vieram da origem.

Não existe atalho de IA para modelo mal documentado. Governança virou requisito de performance da resposta.

Cinco coisas para levar daqui

- 1 Meça antes de mexer. Palpite não é diagnóstico.
- 2 Comece pela camada 1: o que entra pesa mais que como o DAX calcula.
- 3 O Labs gera evidência, o Copilot lê a evidência, a decisão é sua.
- 4 Sem descrição e sem nome de negócio, não existe modelo pronto para IA.
- 5 Transforme o diagnóstico em rotina agendada, não em projeto de fim de ano.

Semantic Link Labs

github.com/microsoft/semantic-link-labs

Documentação da API

semantic-link-labs.readthedocs.io

Direct Lake

learn.microsoft.com/fabric/fundamentals/direct-lake-overview

Modelo pronto para Copilot

learn.microsoft.com/power-bi/create-reports/copilot-evaluate-data

Material desta sessão

base sintética + notebooks + guia passo a passo

MATERIAL DA SESSÃO

Leve tudo com você

Aponte a câmera para o QR Code

Base sintética, notebooks e guia passo a passo · github.com/andrerosa1977/mvpconf2026-tuning-modelos-semanticos



AVALIAÇÃO DA SESSÃO

Avalie esta sessão

Aponte a câmera para o QR Code

Leva menos de um minuto — e o feedback vai direto para a organização e para o palestrante.

MVP CONF 2026



OBRIGADO

Leve o diagnóstico para o seu modelo

André Rosa

Head of BI & Tech Learning · DataEX / IT Forest · Microsoft Certified Trainer



LINKEDIN

linkedin.com/in/andre-rosa77

Aponte a câmera para se conectar



Obrigada(o)!



ANDRÉ ROSA · LINKEDIN

[linkedin.com/in/andre-rosa77](https://www.linkedin.com/in/andre-rosa77)

Aponte a câmera para se conectar